SRE IN GAMING TECH: HANDLING MILLIONS OF REAL-TIME REQUESTS

Siddharth Vijay
Head of DevOps/SRE & AVP - Engg @ Baazi Games

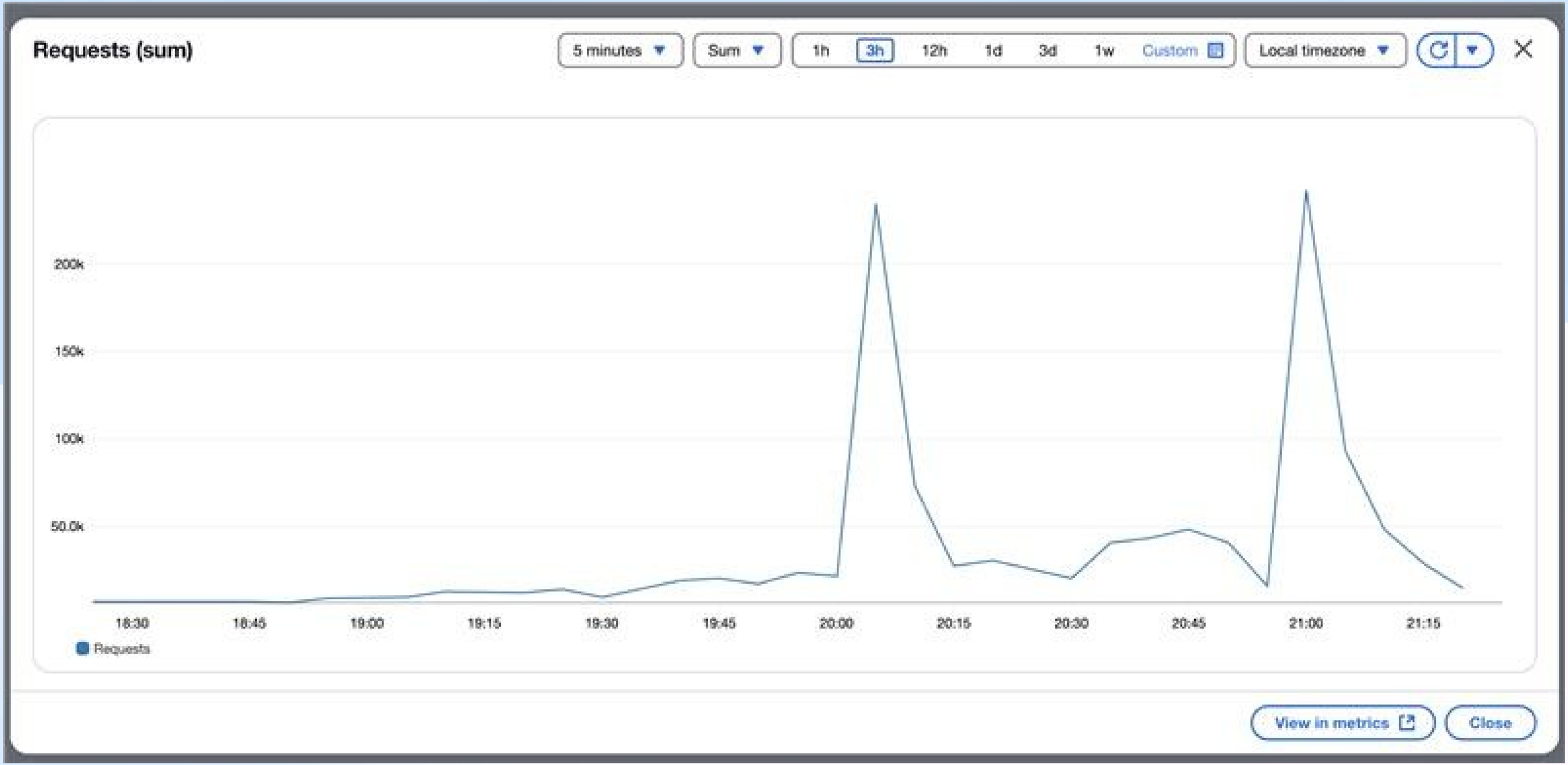
The Stake of Real-Time Gaming

- Modern gaming = real-time interactions, global users, zero tolerance for latency
- A single glitch = thousands of player drop-offs
- Our mission: 99.99% uptime across millions of requests per minute

Low Latency & High Availability

- CDN & Edge Computing: Leveraged AWS CloudFront and Route 53 for static content delivery, reducing latency during peak gaming hours.
- Multi-region Deployments: Transitioned from monolithic setup to a distributed architecture, enabling zero-downtime deployments and regional failovers.
- Read-Replica Databases: Deployed read-replica Aurora/RDS databases across regions for local reads and lower query times.
- Optimized Load Balancing: Configured ALBs and NLBs for multi-region intelligent traffic routing.
- Baazi Medium Link - Transitioning tech architecture to handle increasing scale

CDN Traffic Spike Graph



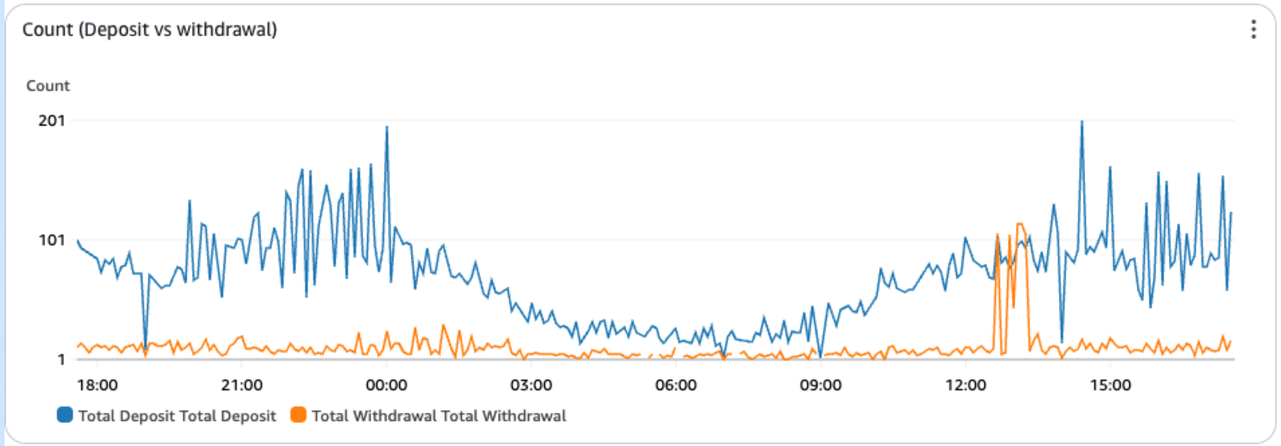
Error Budgets for Game Stability

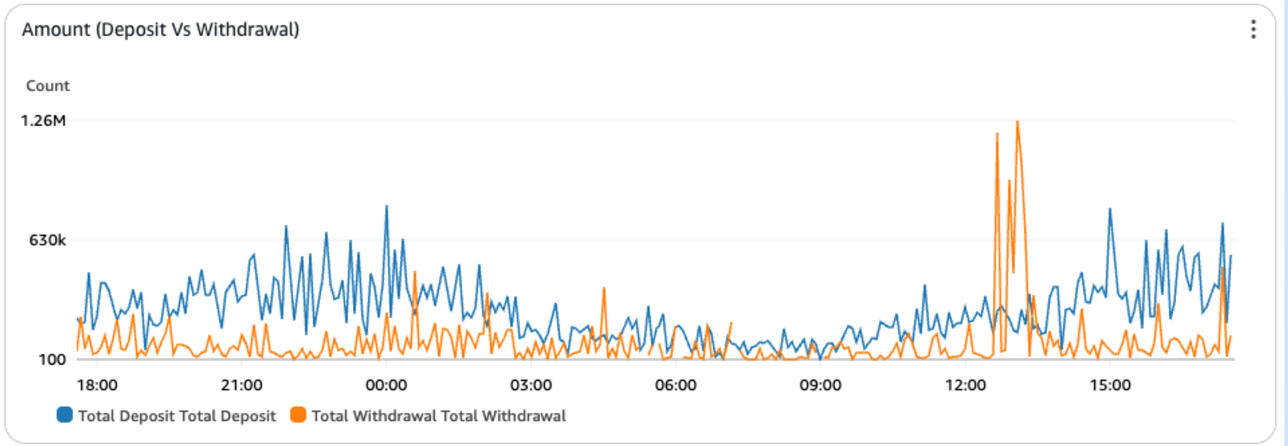
- Defined SLOs using real-time data dashboards for uptime and latency.
- Error Budgets used to inform release velocity—feature rollouts paused if budgets are exceeded.
- SLA Alignment: Error budgets aligned with third-party service SLAs to identify dependency risks.
- Automated Alerts: Triggered alerts when error budget thresholds were at risk of being breached.

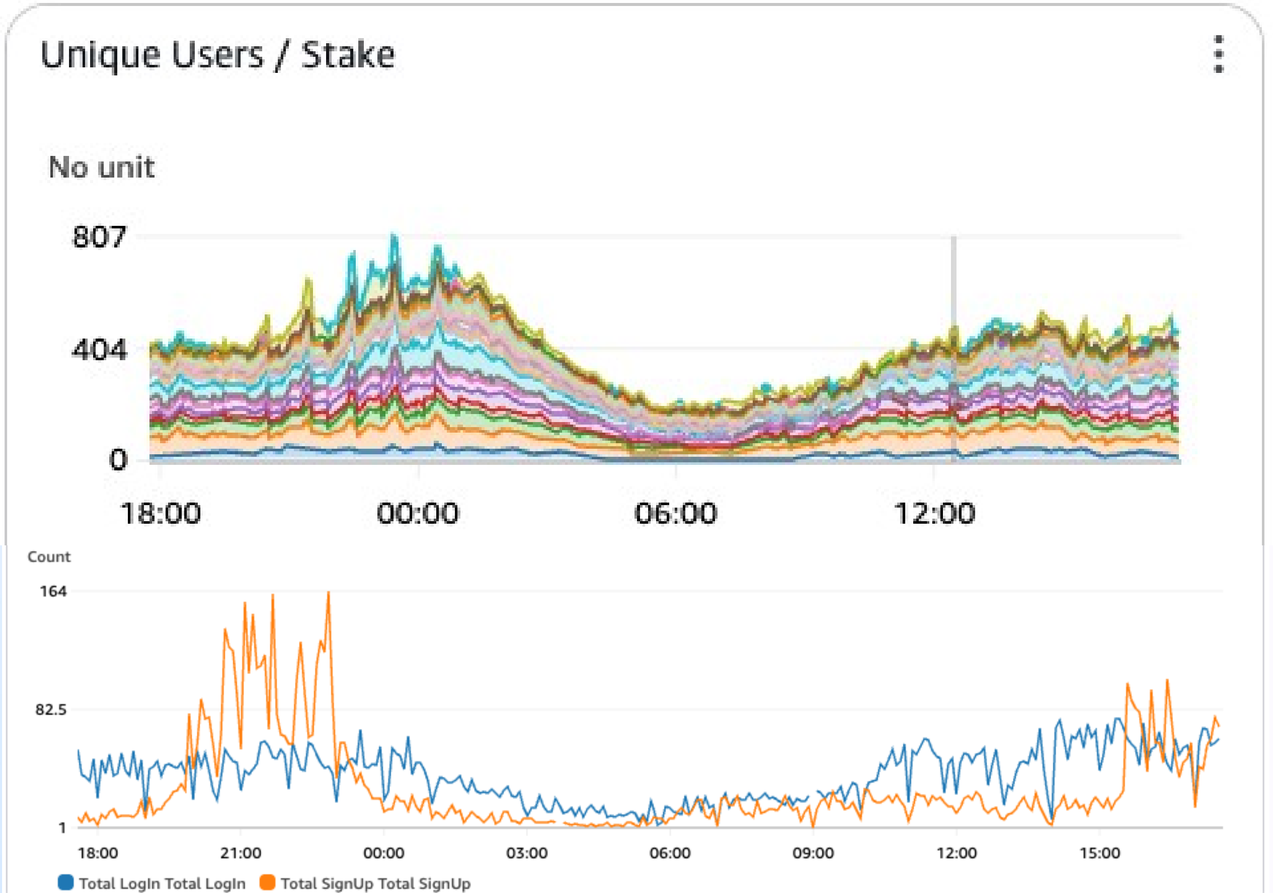
Observability: Logs, Metrics & Traces for Real-Time Insights

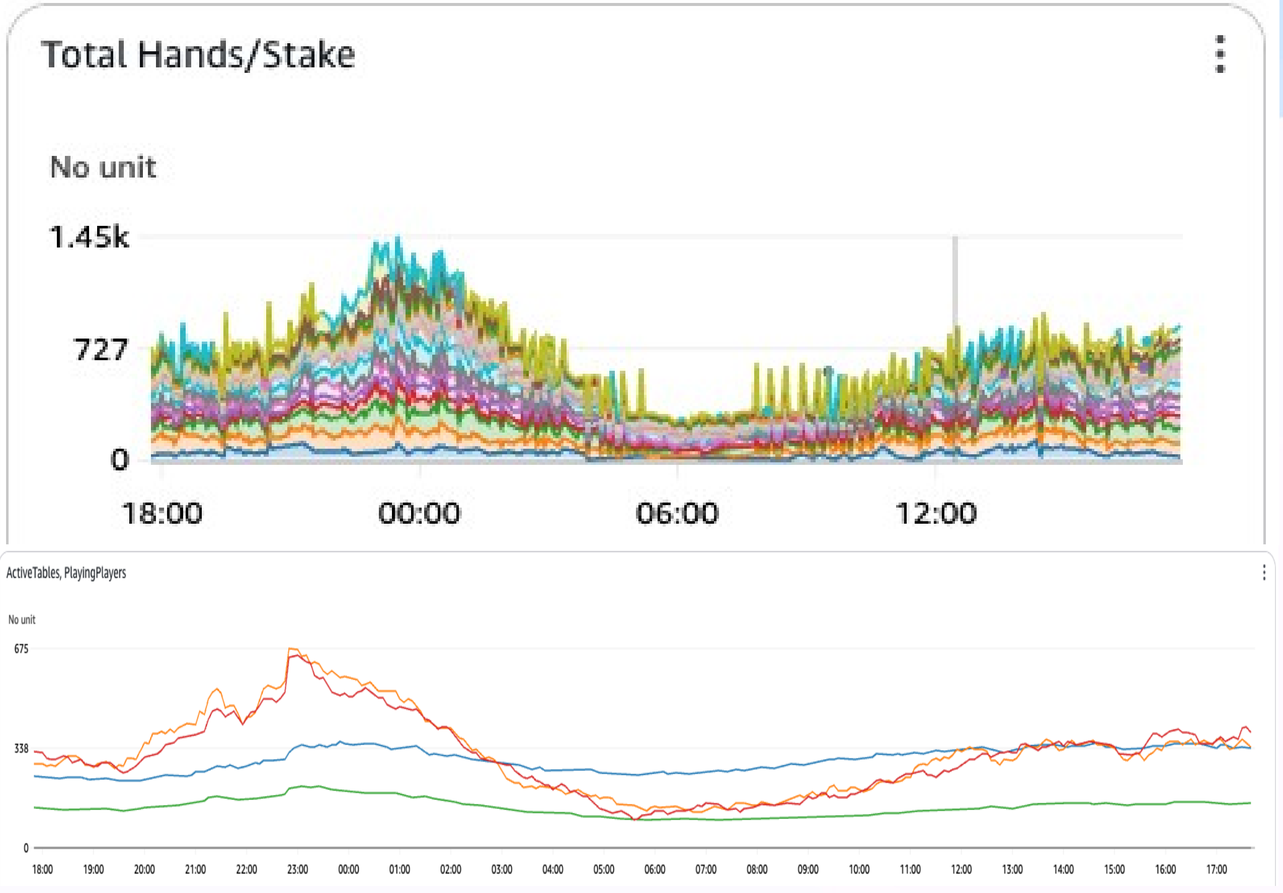
- AWS CloudWatch Dashboards automated with Python for quick insights.
- Microservice Metrics exposed via Istio and visualized with Kiali for deep traffic analysis.
- Real-time monitoring of player actions and server health enables instant alerting and root cause identification.
- Distributed Tracing: Implemented OpenTelemetry for deep insight into service-to-service communication.
- Custom Business Metrics: Monitored player retention, in-game purchases, and transaction volumes.

CloudWatch Dashboard Monitoring Data







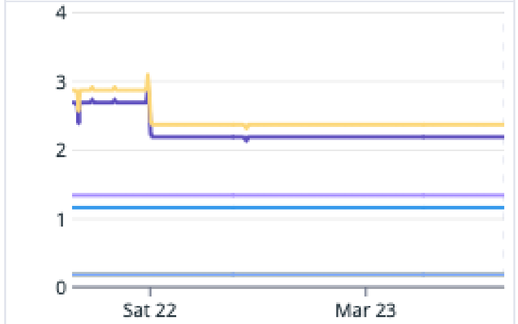


Auto-Scaling Game Servers

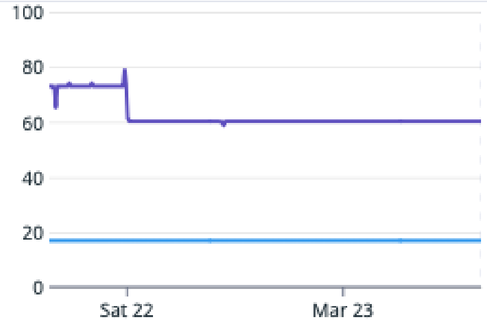
- Kubernetes + Istio used for efficient traffic management and auto-scaling.
- Spot Instances & Fargate combined with predictive models to reduce cost without compromising performance.
- Horizontal Pod Autoscaling (HPA): Applied to critical game pods to dynamically adjust replicas.
- Cluster Autoscaler: Automatically added or removed worker nodes based on traffic.
- Warm Pools: Maintained warm EC2 instances for faster scale-out during spikes.
- Baazi Medium Blog Link -

K8s Karpenter Monitoring Dashboard

CPU Requested



% CPU Requested

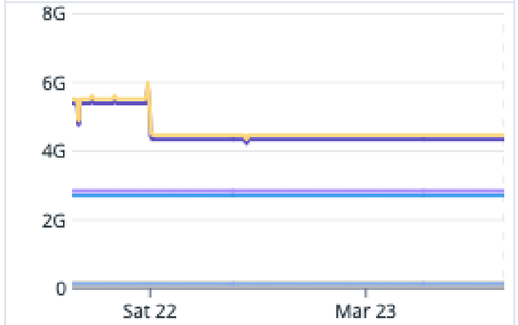


Allocatable CPU

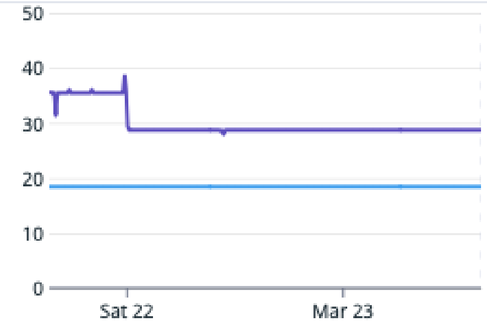
5m

INSTANCE_...	KUBE_CLU...	↓ ALLOC...
c6a.2xlarge	pb-qa-pro...	7.91
m5.xlarge	pb-qa-pro...	3.92

Memory Requested



% Memory Requested

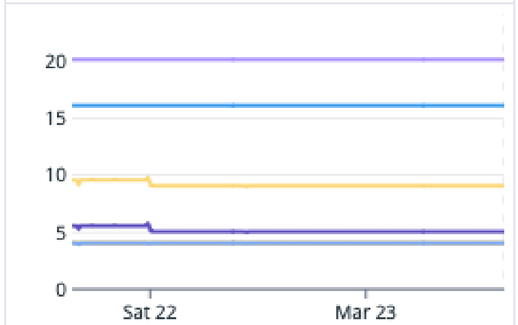


Allocatable Memory

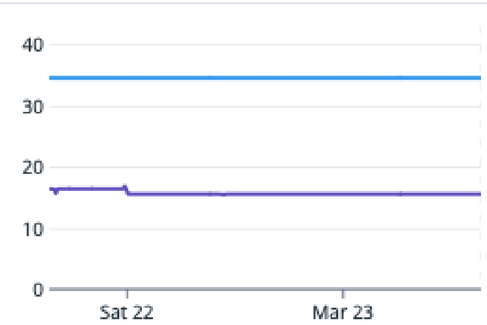
5m

INSTANCE_...	KUBE_CLU...	↓ ALLOC...
m5.xlarge	pb-qa-pro...	15.5G
c6a.2xlarge	pb-qa-pro...	15.3G

Pods Requested



% Pods Requested



Allocatable Pods

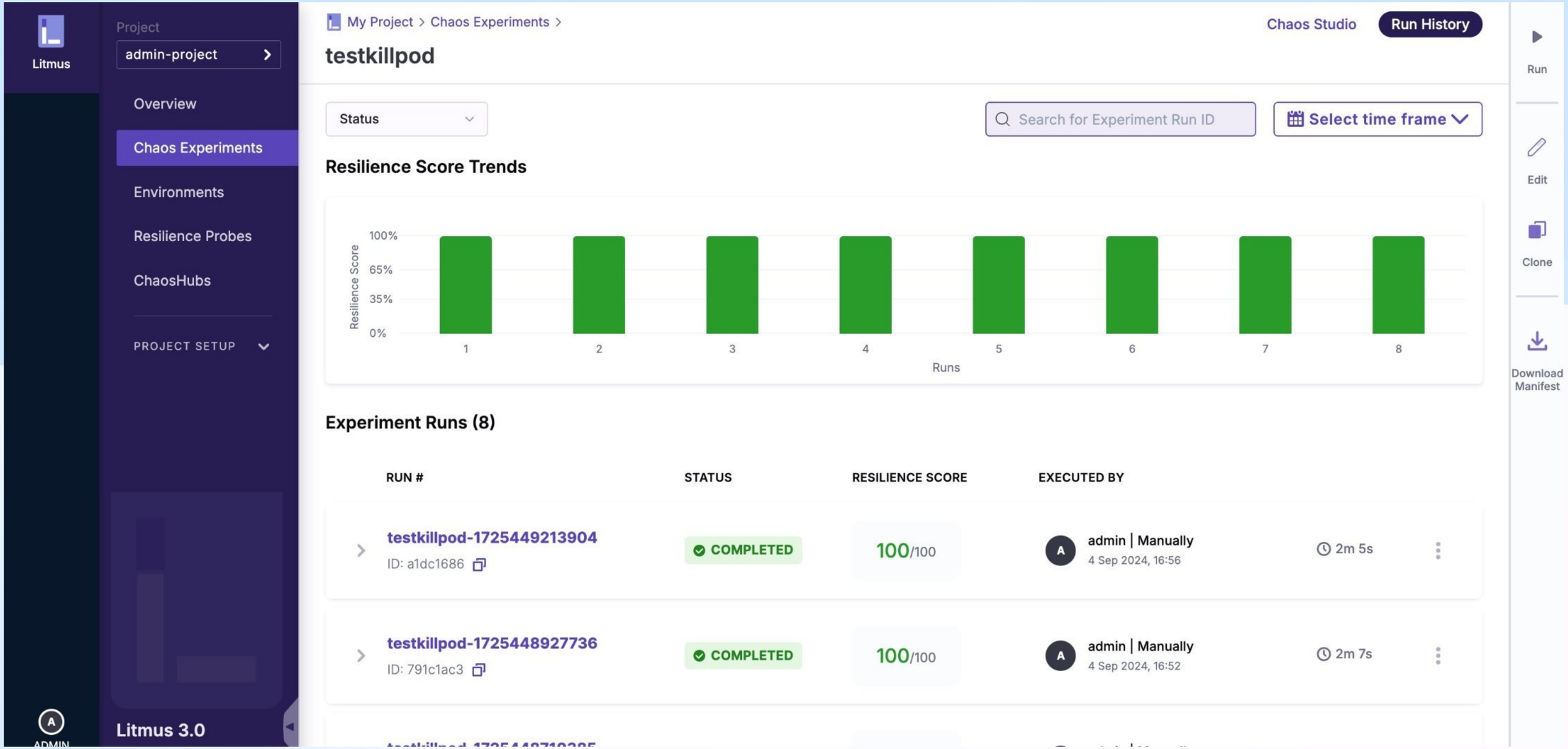
5m

INSTANCE_...	KUBE_CLU...	↓ ALLOC...
c6a.2xlarge	pb-qa-pro...	58
m5.xlarge	pb-qa-pro...	58

Chaos Engineering for Gaming Stability

- Chaos Testing Tools: Simulated failures with Gremlin-like testing internally.
- Failure Scenarios: Included DB overload, server crash, and matchmaking failures.
- Istio Traffic Shifting: Used to test behavior during partial service degradation.
- Controlled Experiments: Conducted during low player load with full rollback capabilities.
- Error Injection: Simulated random latency, dropped packets, and throttling to harden services.

Litmus Dashboard for Chaos Engineering



Blameless Postmortems & Instant Rollbacks

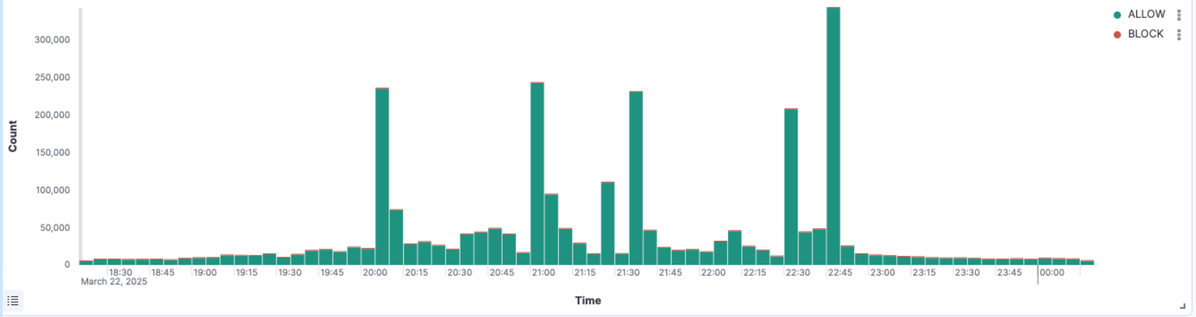
- Postmortems adopted from SRE best practices—root causes identified without blame.
- Feature Flags: Enabled controlled rollouts and instant reversals if instability is detected.
- Incident Timeline Documentation: Captured during postmortems to improve MTTR.
- Learning Repository: Shared findings with engineering and operations teams for continuous improvement.
- Rollback Automation: CI/CD pipelines integrated with flags for rollback within seconds.

Secure & Resilient Gaming Infrastructure

- DDoS Protection: Leveraged AWS WAF and security groups for active threat mitigation.
- Anti-Cheat: Behavior analytics and pattern detection applied for fraud control.
- Transaction Security: Encrypted communication and fraud monitoring integrated with observability stack.
- Zero Trust Network: Applied strict IAM roles and VPC security boundaries.
- Real-Time WAF Updates: Integrated with blocklists via automated IPSet updates using Python and Boto3.
- Baazi Medium Blog Link -

CDN WAF Monitoring Data

[AWS-WAF]Rule_Action



[AWS-WAF]IP_COUNT

2,706,334
Count

294,052
UniqueIP

[AWS-WAF]Client_IP

Client_IP	Country_Name	Count	Count percentages
64.233.173.37	United States	2,722	0.101%
64.233.173.38	United States	2,700	0.1%
64.233.173.39	United States	1,671	0.062%
49.207.63.223	India	1,443	0.053%
64.233.173.36	United States	1,274	0.047%
64.233.173.43	United States	1,209	0.045%
64.233.173.42	United States	923	0.034%
		2,706,334	

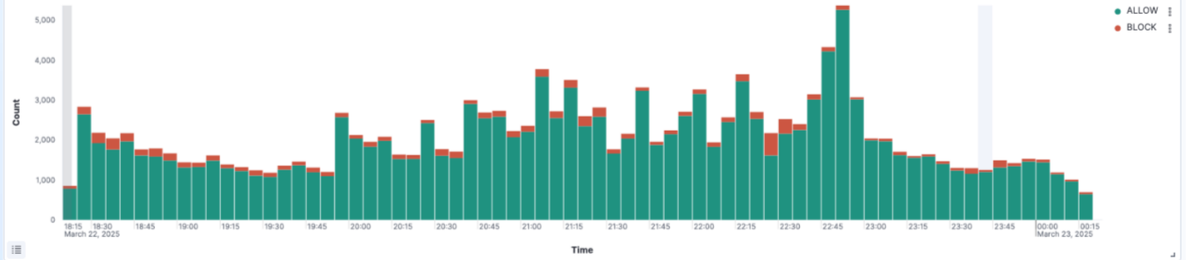
[AWS-WAF]Country-City

Country_Name	City_Name	Count	Count percentages
India	Bengaluru	252,066	9.314%
India	Kolkata	159,783	5.904%
India	Delhi	156,743	5.792%
India	Mumbai	152,596	5.638%
India	Missing	144,905	5.354%
India	Other	1,790,503	66.16%
United States	Missing	19,632	0.725%
		2,706,334	

Oauth WAF Monitoring Data

Create visualization | Add panel | Add from library | Controls

[AWS-WAF]Action_Rule



[AWS-WAF]Unique_IP

151,368
Count

28,871
Unique_IP

[AWS-WAF]Client_ip

Client_IP	Country	Count	Count percentages
217.21.90.178	India	6,178	4.081%
188.213.174.145	Italy	4,886	3.228%
188.213.174.65	Italy	1,941	1.282%
223.233.80.116	India	1,359	0.898%
49.36.183.154	India	1,126	0.744%
92.119.157.91	India	1,098	0.725%
103.39.116.171	India	1,015	0.671%
		151,368	

Rows per page: 10 ‹ 1 2 3 4 5 ··· 12 ›

[AWS-WAF]Country-City

Country_Name	City_Name	Count	Count percentages
India	Mumbai	16,161	10.677%
India	Delhi	11,950	7.895%
India	Missing	8,920	5.893%
India	Hyderabad	8,691	5.742%
India	Bengaluru	8,489	5.608%
India	Other	78,247	51.693%
Italy	Arezzo	7,302	4.824%
		151,368	

Rows per page: 20 ‹ 1 2 ›

Load Testing with Real-World Scenarios

- Simulated Load Testing: Created stress conditions mimicking real-time tournaments using JMeter/k6.
- Network Conditions Testing: Simulated lag, latency, and high-ping environments to ensure client-server sync.
- Game Event Replay: Replayed historical player events at accelerated speed to test stability.
- Region-Based Load Testing: Verified multi-region performance across player clusters.

K6 Load Testing Result on API Endpoint

Total Requests	Failed Requests	Breached Thresholds	Failed Checks
24006	0	0	0

Request Metrics | Other Stats | Checks & Groups

	Count	Rate	Average	Maximum	Median	Minimum	90th Percentile	95th Percentile
http_req_duration	-	-	207.54	1985.91	98.63	40.46	513.88	765.76
http_req_waiting	-	-	206.98	1985.85	98.35	38.50	513.05	764.30
http_req_connecting	-	-	0.84	58.02	-	-	-	-
http_req_tls_handshaking	-	-	15.66	3187.48	-	-	-	-
http_req_sending	-	-	0.07	16.23	0.05	0.01	0.08	0.11
http_req_receiving	-	-	0.49	280.17	0.03	0.01	0.10	0.14
http_req_blocked	-	-	16.50	3231.54	-	-	0.00	0.00
iteration_duration	-	-	226.26	4669.62	101.23	42.49	521.80	775.51

Automation-Driven SRE for Smarter Operations

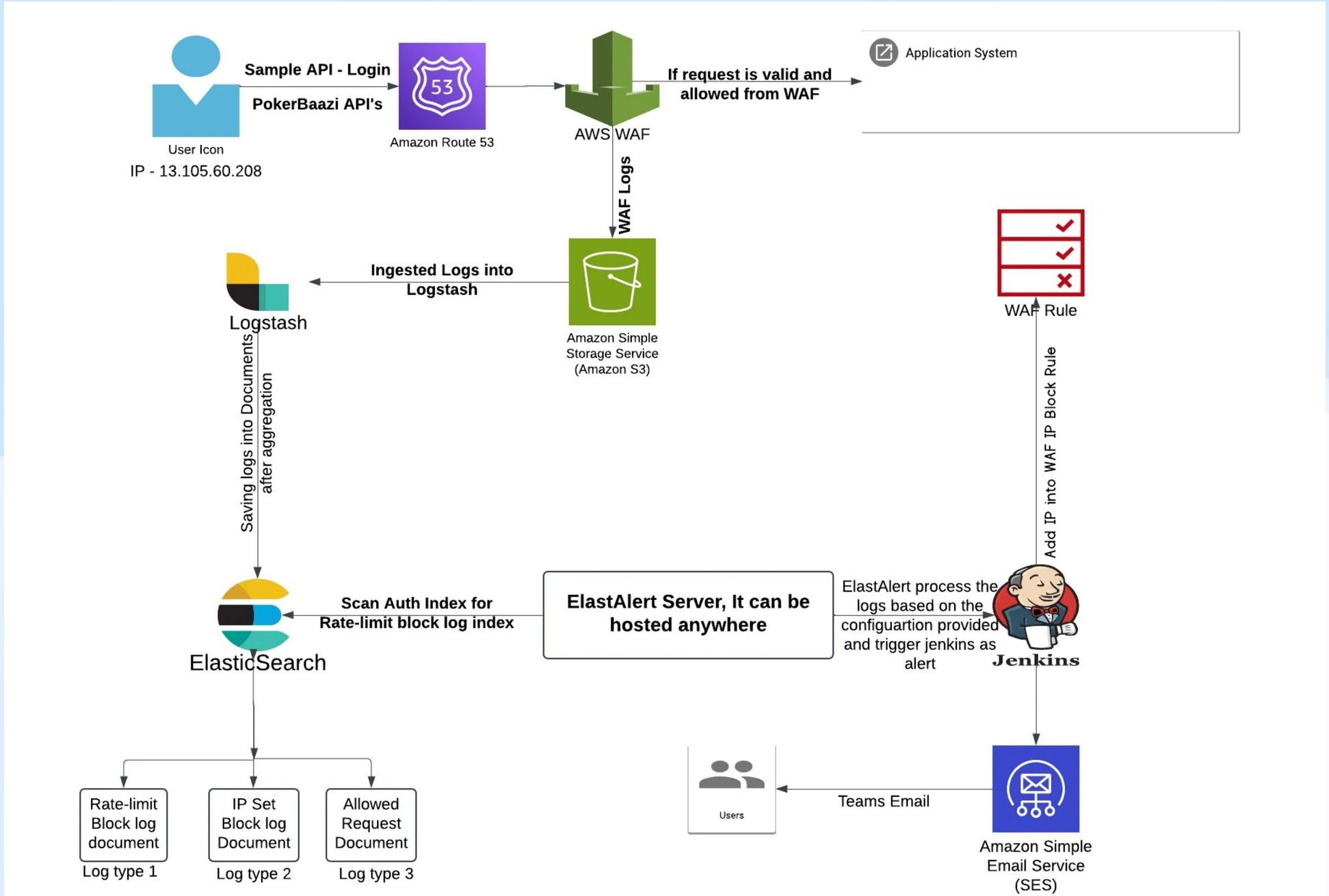
- Updation of Cloudwatch dashboard with new infra using python boto3 library.

- Automatically Blocking malicious ip using elastalert, python and Jenkins.

- Anomaly Detection: Integrated Datadog, Cloudwatch to detect anomaly patterns in application logs as well as infrastructure logs.

- Baazi Medium Blog Link - Streamlining AWS CloudWatch Dashboards

AutomatedBlocking Malicious IP Using Elastalert



Final Takeaway: SRE in Gaming = Uptime + Speed + Scale

- The Goal: Deliver seamless, uninterrupted, and fair gaming experiences for millions of players.
- The Strategy: Automate, observe, secure, and stress-test everything.
- The Mindset: Proactively prevent issues before they affect players.